TD 02 : Encapsulation et Méthodes redéfinies
(2h)

Consignes :

- ⇒ Ce travail est INDIVIDUEL
- ⇒ Chaque étudiant doit répondre à TOUS les exercices
- ⇒ **L'utilisation d'un IDE est INTERDITE**
- ⇒ Supports de cours à consulter : CM1, CM2

Acquis d'apprentissage :

A la fin de ce TD, chaque étudiant doit :
- Comprendre l'intérêt de gérer la visibilité des attributs et des méthodes
- Apprendre la syntaxe des accesseurs / mutateurs et leurs utilités
- Comprendre l'usage de la méthode toString et l'intérêt de la redéfinition de méthodes
- Comprendre l'usage de la méthode equals et son intérêt

À FAIRE SUR PAPIER !!!
N'utilisez pas d'IDE, le but n'est pas de donner le résultat affiché à l'écran mais de comprendre le fonctionnement

Exercice 1

Considérons la classe Point suivante :

```
class Point {
    int x;
    int y;

    void afficher() {
        System.out.print("(" + x + " " + y + ")");
    }

    String tooString() {
        String s = "(" + x + " " + y + ")";
        return s;
    }

    Point translation(int dx, int dy) {
        Point resultat = new Point();
        resultat.x = x + dx;
        resultat.y = y + dy;
        return resultat;
    }

    public double distance (Point p) {
        return Math.sqrt((this.x - p.x) * (this.x - p.x) + (y - p.y) * (y -
p.y));

    }
}
```

1. Rendre privés les attributs de la classe Point

2. Lui ajouter un constructeur à deux paramètres. Assurez-vous que le constructeur par défaut continue à s'exécuter.
3. Écrire les getters (accesseurs) et setters (mutateurs) relatifs aux attributs
4. Modifier les **lignes en rouge** dans la **méthode main** ci-dessous de façon à les corriger :

```
public class TestPoint
{
    public static void main (String[] args)
    {
        Point p = new Point();
        System.out.println(p.x + " " + p.y);

        p.afficher();
        System.out.println();

        p.x = 3;
        p.y = 8;

        p.afficher();
        System.out.println();

        String s = p.tooString();
        System.out.println(s);

        System.out.println(p.tooString());

        Point q;
        q = p.translation(100, 200);
        q.afficher();

    }
}
```

a) Que va afficher le programme ci-dessus à l'écran après modification ?

b) Que faut-il ajouter pour automatiquement afficher l'objet Point comme une chaîne de caractères ? Quelles autres modifications cela entraine-t-il ?

c) Modifier le programme main en conséquence.

d) Que va afficher cette instruction si elle est ajoutée à la fin du programme **main** System.*out*.println(p == q) ; expliquer pourquoi ?

Exercice 2

Considérons la classe ci-dessous, qui utilise la classe **Point** (après modification) de l'exercice 1.

```
class Segment {

   Point a;
   Point b;

   Segment translation (int dx, int dy) {
      Segment resultat = new Segment ();

      resultat.a = a.translation(dx, dy);
      resultat.b = b.translation(dx, dy);

      return resultat;
   }
}
```

1. Rendre privés les attributs de la classe **Segment**
2. Écrire le constructeur à deux paramètres (la même condition que précédemment doit être respectée)
3. Écrire les getters / setters relatifs aux attributs
4. Modifier la méthode **main** ci-dessous et compléter la classe **Segment** afin que le programme puisse s'exécuter correctement.

```
public class Main
{
    public static void main(String[] args)
    {
        Segment segment = new Segment();

        Point a = new Point();
        Point b = new Point();

        a.x = 10;
        a.y = 20;
        b.x = 30;
        b.y = 40;

        segment.a = a;
        segment.b = b;

        System.out.println(segment);


        Segment translate;
        translate = segment.translation(100, 200);

        System.out.println(translate);

    }
}
```

5. Ajouter à la classe **Segment** un constructeur de copie
6. Écrire une méthode **longueur** calculant la longueur d'un segment
7. Compléter la méthode main
   a) En créant un segment **segCop** copie du segment **segment**
   b) En l'affichant
   c) En affichant la longueur de ce segment
8. Que vont afficher les deux instructions suivantes ? Expliquer.

```
System.out.println(segment == segCop);
System.out.println(segment.getA() == segCop.getA());
```

9. Dans la méthode main, créer un tableau « **segments** » de segments dans lequel sont rangées les 3 instances de la classe Segment précédentes
10. Afficher les segments du tableau en utilisant une boucle for traditionnelle.
11. Afficher les segments du tableau en utilisant une boucle for-each (voir le complément ci-après)

Complément : La boucle for-each

Exemple pour un tableau d'objets de type String :

```
String[] names = {"Alice", "Bob", "Charlie", "Diana", "Eve", "Jean"};
for (String name : names) {
        System.out.println(name);
}
```

- Notez bien la convention de nommage : singulier (**name**) pour la variable de boucle et pluriel (**names**) pour le tableau.
- En Java, lors du parcours du tableau (**names**) contenant des objets de type **Type** (ici **String**), un objet (**name**) de type **Type** (**String**) peut servir de variable de boucle. Cela permet d'éviter d'utiliser un indice entier pour accéder aux éléments du conteneur.

Exercices supplémentaires

Exercice 1

Opérateur == VS la méthode equals() avec le Jeu des Identités Secrètes

1. Anticipez et expliquez le résultat de l'exécution du programme en prenant en compte les entrées suivantes :

- ***a.*** Entrée : "Agent007", "Agent007"
- ***b.*** Entrée : "Agent007", "Agent007" en décommentant la ligne ***identite2=identite1;***
- ***c.*** Entrée : "Agent007", "Agent007" en décommentant la ligne ***identite2=new String(identite1);***
- ***d.*** Entrée : "Agent007", "Agent008"
- ***e.*** Entrée : "Agent007", "Agent008" en décommentant la ligne ***identite2=identite1;***
- ***f.*** Entrée : "Agent007", "Agent008" " en décommentant la ligne ***identite2=new String(identite1);***

```
import java.util.Scanner;
public class JeuIdentitesSecretes {
        public static void main(String[] args) {
            String identite1;
            String identite2;
            Scanner scanner = new Scanner(System.in);
            System.out.println("Bienvenue au jeu des Identités
Secrètes !");
            System.out.println("Deux suspects ont fourni des
identités secrètes.");
            System.out.println("Votre mission est de déterminer si
ces identités sont identiques.");
            System.out.print("Veuillez entrer l'identité secrète du
premier suspect : ");
            identite1 = scanner.nextLine();
            System.out.print("Veuillez entrer l'identité secrète du
deuxième suspect : ");
            identite2 = scanner.nextLine();
            //identite2=identite1;
            //identite2=new String(identite1);
            boolean identitesIdentiquesOp = (identite1 == identite2);
            boolean identitesIdentiquesEq =
identite1.equals(identite2);
```

```
            System.out.println("\nRésultats de l'enquête :");
            System.out.println("Comparaison avec '==': " +
(identitesIdentiquesOp ? "Les identités sont identiques." : "Les
identités ne sont pas identiques."));
            System.out.println("Comparaison avec 'equals()': " +
(identitesIdentiquesEq ? "Les identités sont identiques." : "Les
identités ne sont pas identiques."));
            scanner.close();
    }
}
```

2. Quelle distinction existe-t-il entre l'utilisation de l'opérateur == et la méthode equals() ?
3. La méthode equals() produira-t-elle le même résultat lorsqu'elle est utilisée avec des objets d'autres types que String ?

Exercice 2

Partie 1

1. Écrire une classe **Auteur** représentant un auteur par son prénom, son nom et sa nationalité.
2. Écrire un constructeur à 3 paramètres.
3. Écrire des accesseurs et des mutateurs pour tous les attributs.
4. Écrire une méthode permettant de convertir un objet de type **Auteur** en une chaîne de caractères de la forme (méthode de représentation textuelle d'un objet) :

   `Auteur :  Prénom - Nom - Nationalité`

Partie 2

1. Écrire une classe **Livre** définissant un livre par :
   - ISBN (String)
   - un auteur
   - un titre (String)
   - une année de sortie (entier non signé)
   - un nombre de pages (entier non signé)
2. Écrire un constructeur à 5 paramètres.
3. Écrire un constructeur de copie.
4. Écrire des getters / setters pour tous les attributs.
5. Écrire une méthode permettant de convertir un objet de type **Livre** en une chaîne de caractères de la forme :

```
ISBN :
Titre :
Année de sortie :
```

```
Nombre de pages :
Auteur :  Prénom - Nom - Nationalité
```

Partie 3

1. Écrire une classe **Librairie** définie comme étant un tableau de livres et la première place libre dans ce tableau.
2. Écrire un constructeur permettant d'avoir **n** livres dans la librairie (**n** donné en paramètre).
3. Écrire une méthode d'instance **ajouter** qui prend en paramètre un livre et qui l'ajoute à la librairie.
4. Écrire la méthode **toString** qui permet de convertir les livres de la librairie en une chaîne de caractères.
5. Écrire la méthode **afficherParAuteur** qui permet d'afficher tous les livres écrits par un auteur donné en paramètre.
6. Écrire une méthode d'instance **insererLivreParAnnee** qui permet d'ajouter un livre à la librairie dans l'ordre croissant selon son année de sortie.

Partie 4

Dans la classe **Main**, tester les méthodes des classes précédentes de la manière la plus exhaustive possible.

Il convient entre autres de :

1. Tester d'abord le cas d'ajout d'un livre sans tenir compte de l'année de sortie
2. Puis, tester le cas où livres sont tous ajoutés selon leurs années de sortie.